# Convolutional pattern matching, the failure function, and a real-valued bad-character rule

Advanced Algorithms — exam project

Luca Simonetti

April 2026

**Abstract**

This project studies the relationship between 1D convolutional neural networks (CNNs) and exact-string-matching algorithms in two directions. **(D1)** Are the weights a CNN learns assimilable to a KMP failure function *sp* or a Z-array? **(D2)** Can one preprocess the pattern (now realised as a learned filter) into a structure that yields a non-constant stride during the scan, the way KMP and Boyer–Moore use their preprocessing tables? We answer both empirically and algorithmically on a minimal setup ($\Sigma = \{A, B, C, D\}$, kernel size $m = 5$, two patterns chosen so that the failure function is non-trivial). For D1 we prove a clean algebraic identity holding for every single-filter CNN on one-hot input and verify, with a numerical residual of $3.8 \times 10^{-6}$ over 9 200 windows, that the per-window logit decomposes as $\text{logit}(W) = \pi - \sum_i \text{pen}[i, W[i]]$, where pen is a real-valued, position-aware generalisation of the Boyer–Moore bad-character table. We further show that even a $K = 4$ ReLU multi-filter network on the same task collapses to the same additive structure ($R^2 \geq 0.987$ across seeds). For D2 we use pen as actual algorithmic preprocessing: a generalised bad-character shift table built from pen delivers a $4.06\times$ reduction in elementary operations on a 20 000-character text, with no loss of recall. We discuss why *sp* itself does not emerge in the weights (the stateless / stateful divide; [5]), and connect the framework to approximate matching.

## 1 Introduction

KMP [1] turns the naive $\mathcal{O}(nm)$ pattern-matching scan into $\mathcal{O}(n)$ by precomputing in $\mathcal{O}(m)$ time the *failure function* $sp[i]$ = length of the longest proper prefix of $P[0..i]$ that is also a suffix; on a mismatch at pattern position $j$ KMP shifts the alignment by $j - sp[j-1]$ instead of by 1. Boyer–Moore [2] precomputes in $\mathcal{O}(m + |\Sigma|)$ time a *bad-character* table $bc[c]$ = rightmost position of the character $c$ in $P$ (or $-1$), and shifts by $\max(1, j - bc[c])$. Both are honest preprocessings of the pattern, both store the preprocessing in a small data structure, both answer their query in $\mathcal{O}(1)$. The Z-array [3] is closely related and serves the same role from a different angle.

A 1D convolution slid over a sequence is mathematically the same operation as scanning a pattern, with the kernel playing the role of the pattern. This obvious analogy invites two natural questions, made the explicit subject of this project:

**D1** Are the weights a CNN learns assimilable to a failure function or a Z-array?

**D2** Can one preprocess the pattern (now realised as a learned filter) into a structure that gives the scan a non-constant stride, the way KMP and BM do with their tables?

We address both. For D1, the answer is *no* for *sp* but *yes* for a different and arguably more natural structure: a real-valued position-aware bad-character table, learned from data. For D2, we use that structure as the preprocessing of an actual matching algorithm and measure its effect.

**Outline.** Section 2 fixes notation and recalls KMP, BM, the Z-array, and the convolution-as-template-matching identity. Section 3 describes the experimental setup. Section 4 reports what a minimal CNN learns and contrasts it with the ground-truth PWM. Section 5 states and verifies the penalty-table identity, and shows that even a multi-filter ReLU network collapses to it. Section 6 positions the penalty table next to *sp* and *bc* as a comparable preprocessing data structure. Section 7 sketches the connection to approximate matching. Section 8 gives the prototype for direction D2: penalty-driven early exit, plus a generalised bad-character shift table, with measured speedup. Sections 9–11 discuss the result, limitations and outlook.

## 2 Preliminaries

Let $\Sigma$ be a finite alphabet, $T \in \Sigma^n$ a text and $P \in \Sigma^m$ a pattern with $m \leq n$. *Exact matching* asks for all positions $s$ such that $T[s..s+m-1] = P$.

**KMP.** The failure function $sp \in \mathbb{Z}_{\geq 0}^m$ is defined by $sp[i] = \max\{k < i+1 : P[0..k-1] = P[i-k+1..i]\}$ (with $sp[0] = 0$). KMP scans $T$ from left to right while maintaining a pattern offset $k$ that is reset to $sp[k-1]$ on a character mismatch. The total cost is $\mathcal{O}(n+m)$ with $\mathcal{O}(m)$ preprocessing and $\mathcal{O}(m)$ storage. Computing $sp$ takes $\mathcal{O}(m)$ time [1, 3].

**Boyer–Moore (bad-character only).** The bad-character table $bc \in \mathbb{Z}^{|\Sigma|}$ stores $bc[c] = \max\{i : P[i] = c\}$ (or $-1$). When a mismatch occurs at pattern position $j$ on text character $c$, BM shifts the alignment by $\max(1, j - bc[c])$. The bad-character rule alone has $\mathcal{O}(nm)$ worst-case complexity but is famously fast in practice for large alphabets [2].

**Z-array.** $Z[i]$ = length of the longest substring of $P$ starting at $i$ that matches a prefix of $P$ (with $Z[0] = m$). The Z-array can be computed in $\mathcal{O}(m)$ and used for matching in $\mathcal{O}(n+m)$ [3].

**Convolution as template matching.** A 1D convolution (more precisely a cross-correlation, as implemented in all major deep-learning frameworks) of a length-$T$ sequence $x$ with a length-$m$ kernel $w$ produces, at every position $s$, the inner product $\sum_{i=0}^{m-1} w_i\, x_{s+i}$. When $x$ is a sequence of one-hot vectors over $\Sigma$ and $w$ is a matrix $w \in \mathbb{R}^{m \times |\Sigma|}$ with one row per kernel position and one column per alphabet character, the same expression becomes

$$\text{logit}(W) \;=\; \sum_{i=0}^{m-1} w[i, W[i]] \;+\; b, \tag{1}$$

which is exactly the score of a Position-Specific Scoring Matrix (PSSM) applied to the window $W$. The connection between convolution and string matching at this level, including the FFT-based exact and wildcard variants, has been studied in classical algorithmics [4].

**Single-filter CNN model.** We use the simplest 1D-CNN that can attempt the task: a single filter $w \in \mathbb{R}^{m \times |\Sigma|}$ and a scalar bias $b$. Per-position logit is (1), the per-position prediction is $\sigma(\text{logit})$, and the training loss is binary cross-entropy with positive-class re-weighting.

**Multi-filter CNN model.** We also use a $K$-filter version with ReLU and a 1×1 mixing layer:

$$\text{logit}(W) = \sum_{k=1}^{K} W_k^{\text{out}} \cdot \max(0,\, \textstyle\sum_i W_k[i, W[i]] + b_k^{\text{conv}}) + b^{\text{out}}. \tag{2}$$

This model is genuinely non-linear in principle; we will see in Section 5 that on our task it is empirically indistinguishable from a single PSSM scorer.

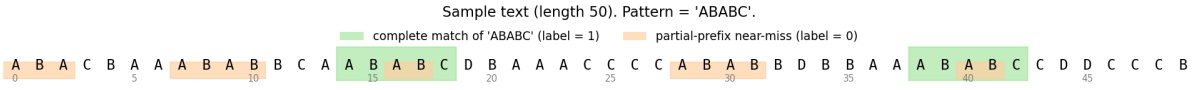


Figure 1: An annotated sample text of length 50. Green spans are complete matches of the pattern (label 1); orange spans are partial-prefix near-misses inserted by the generator (label 0). Token boundaries can create accidental matches and near-misses, which are captured correctly by KMP-based labelling.

## 3 Setup

**Alphabet.** $\Sigma = \{A, B, C, D\}$, $|\Sigma| = 4$.

**Patterns.** Two choices, both with non-trivial failure function:

- `ABABC`: $sp = [0, 0, 1, 2, 0]$, $Z = [5, 0, 2, 0, 0]$. Final character is disambiguating; complete matches do not auto-overlap.
- `ABABA`: $sp = [0, 0, 1, 2, 3]$, $Z = [5, 0, 3, 0, 1]$. Strongly auto-overlapping: `ABABABA` contains two complete matches at offsets 0 and 2.

**Texts.** Length $T = 50$ (or $T = 20\,000$ for the D2 prototype of Section 8). Each text is a shuffled concatenation of:

- $n_{\text{complete}}$ exact occurrences of $P$;
- $n_{\text{partial}}$ deliberate near-miss tokens of the form $P[:k] \cdot c$ with $c \neq P[k]$ for $k \in \{1, \ldots, m-1\}$ — exactly the windows where naive matching wastes work and where KMP's failure function earns its keep;
- uniform-random filler characters padding the rest.

Token order is shuffled, so token boundaries can produce additional accidental near-misses or matches; labels are recovered *after* shuffling by running KMP on the resulting text. Figure 1 shows a typical sample.

**Training.** Adam, learning rate $10^{-2}$, batch size 64, 80 epochs for the single-filter model and 150 for the multi-filter one; positive-class weight 20 to counter the $\sim$5% positive rate. JAX, CPU.

## 4 Direction 1: what a single filter learns

Trained on `ABABC`, the single filter recovers the pattern in its row-wise argmax (`ABABC`) but with a *signed* shape: the right character at each position has a moderate positive weight ($\approx +2$ to $+3.5$) while wrong characters carry *negative* weights ($\approx -2.5$ to $-4.4$). Figure 2 compares the learned filter with the ground-truth one-hot PWM. With this filter and threshold 0 on the logit, validation metrics are F1 = 0.97, recall 1.00, precision 0.94.

The validation logits split cleanly (Figure 3). Every true match achieves the same score +4.96 (the dot product of $w$ with the one-hot pattern is constant); non-matches are spread well below zero. The 63 false positives at threshold 0 are *all* 1-mismatch windows (`ABBBC`, `BBABC`); their logit is just barely positive (median +0.33). They are not "overshoots" but the bias being slightly miscalibrated for these specific shapes.

Replacing the trained filter by min-max-per-row, min-max-global, ReLU+row-max, or *the pure one-hot ground-truth PWM* — and sweeping the threshold — all reach F1 = 1.000. The signed shape is therefore not *necessary*: an additive PWM with calibrated threshold solves the task. The signed shape is what optimisation *picks*, presumably because the gradient signal is denser there.

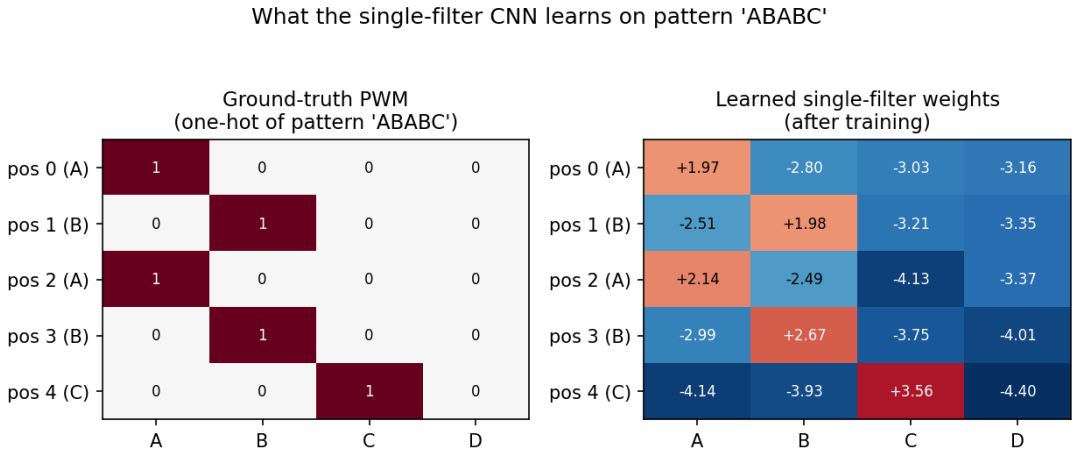


Figure 2: Ground-truth PWM (left) and learned single-filter weights (right) for `ABABC`. The argmax of each learned row equals the pattern character; off-diagonal cells carry negative weight, encoding mismatch *penalties*.

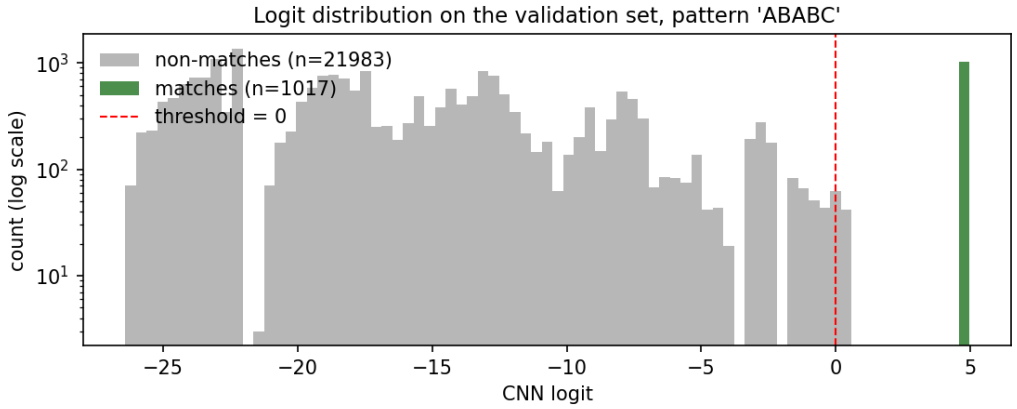


Figure 3: Validation logit distribution (log scale) on `ABABC`. Matches concentrate at one point; non-matches form a broad distribution well below zero. Threshold 0 separates the two regimes with high but imperfect margin.

# 5 The structure that emerges in place of *sp*

## 5.1 The penalty-table identity

Given the trained filter $w$ and the recovered pattern $P_i = \arg\max_c w[i, c]$, define the *penalty table* $\text{pen} \in \mathbb{R}^{m \times |\Sigma|}$:

$$\text{pen}[i, c] \;=\; w[i, P_i] - w[i, c] \;\geq\; 0, \qquad \text{pen}[i, P_i] = 0. \tag{3}$$

Let $\pi = b + \sum_i w[i, P_i]$ be the *pattern logit* (the score the model assigns to a perfect match). Then for every window $W$ of length $m$ over $\Sigma$:

$$\boxed{\text{logit}(W) \;=\; \pi \;-\; \sum_{i=0}^{m-1} \text{pen}[i, W[i]]} \tag{4}$$

*Proof.* Substitute $w[i, W[i]] = w[i, P_i] - \text{pen}[i, W[i]]$ in the dot product and collect constants: $\sum_i w[i, W[i]] + b = \sum_i (w[i, P_i] - \text{pen}[i, W[i]]) + b = \pi - \sum_i \text{pen}[i, W[i]]$. □

Figure 4 shows the penalty tables learned for the two patterns. We verified (4) numerically across 9 200 validation windows per pattern; the maximum absolute discrepancy between $\text{logit}(W)$

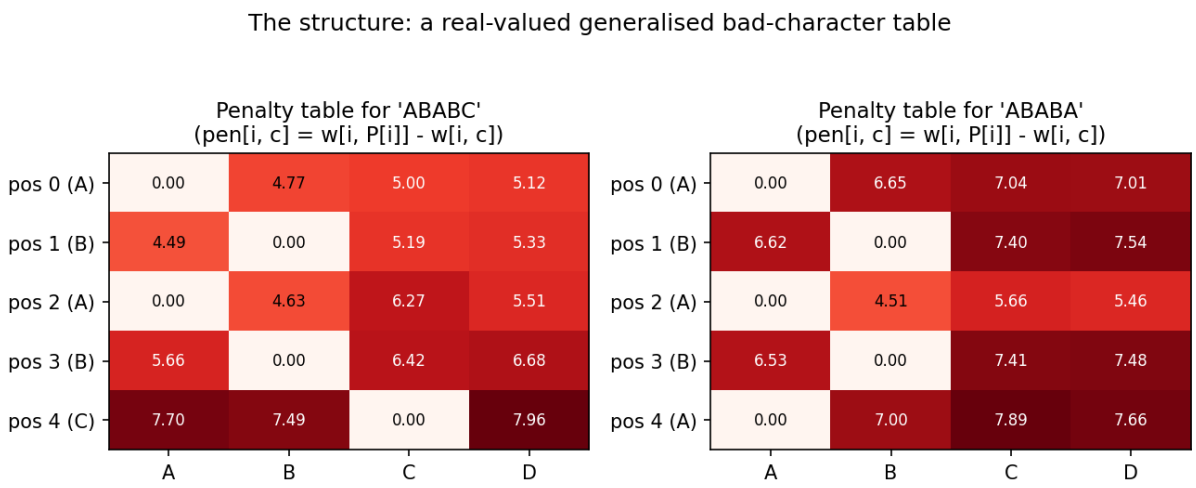


Figure 4: Penalty tables for the two patterns. Cells are $\geq 0$; the zero diagonal corresponds to $w[i, P_i]$. On `ABABC` position 4 carries the largest penalties (the disambiguating `C`); on `ABABA` position 2 (the most auto-correlated row) carries *smaller* penalties — the network is statistically less confident there.

as computed by the CNN and the right-hand side of (4) is $3.8 \times 10^{-6}$, i.e. floating-point noise (Figure 5).

### 5.2 Even the multi-filter model collapses to a PSSM

A natural objection to (4) is that it is an artefact of using a single linear filter. To test this we trained the $K = 4$ multi-filter model of (2) on `ABABC` and asked whether its logit could be reproduced by a single additive PSSM $c_0 + \sum_i S[i, W[i]]$ fit by least squares to the validation logits. On the seed shown in Figure 6 the fit gives $R^2 = 1.0000$, residual standard deviation 0.01 against target standard deviation 23.4, maximum residual 0.04; across random seeds it ranges $R^2 \in [0.987, 1.000]$. The collapse is therefore approximate rather than an exact identity, but on this task the multi-filter ReLU network is in every case almost indistinguishable from a PSSM scorer.

Why does a genuinely non-linear network behave additively? Not because the non-linearity is dormant: we measured that roughly a quarter of the pre-ReLU activations are negative, and hence clipped by the ReLU. The additivity is instead an empirical property of this task distribution — the anti-pattern-detecting filters, clipping included, happen to combine on the windows the generator produces into a per-position additive score to within the residual reported above. We do not claim this holds off-distribution; Section 10 returns to the point. The model is non-linear in principle and additive in practice on this task.

### 5.3 A faint distributional signature of *sp* in the margin

Define the per-position margin $\text{margin}[i] = w[i, P_i] - \max_{c \neq P_i} w[i, c] = \min_{c \neq P_i} \text{pen}[i, c]$.

On `ABABA` (Figure 7) the position with the lowest margin (4.51) is 2, which is also the first row where *sp* becomes positive — exactly the position at which the pattern starts overlapping itself. On `ABABC` the relationship inverts (the highest-*sp* row has the largest margin, because our deliberate near-miss tokens of types `ABA?` and `ABAB?` deposit dense negative signal there). Pearson correlations across the ten (pattern, position) pairs are weak: $\rho(sp, \text{margin}) = +0.18$, $\rho(Z, \text{margin}) = -0.30$. The signature of pattern auto-correlation in the weights is real but distribution-dependent and non-monotone in *sp*. We record it as an observation, not as a theorem.

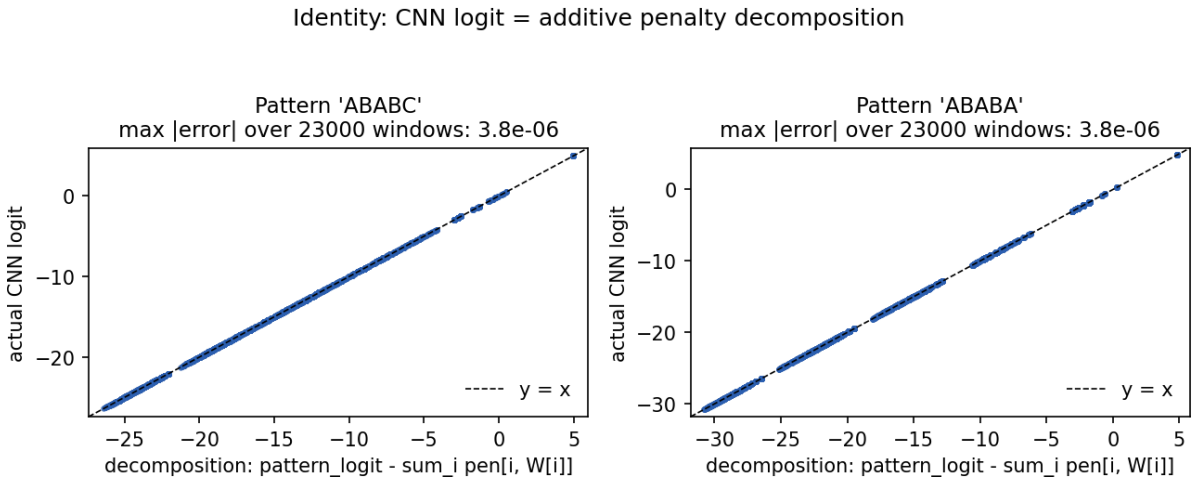


Figure 5: Numerical verification of identity (4). Every validation window lies on the line $y = x$ to floating-point precision, on both patterns.

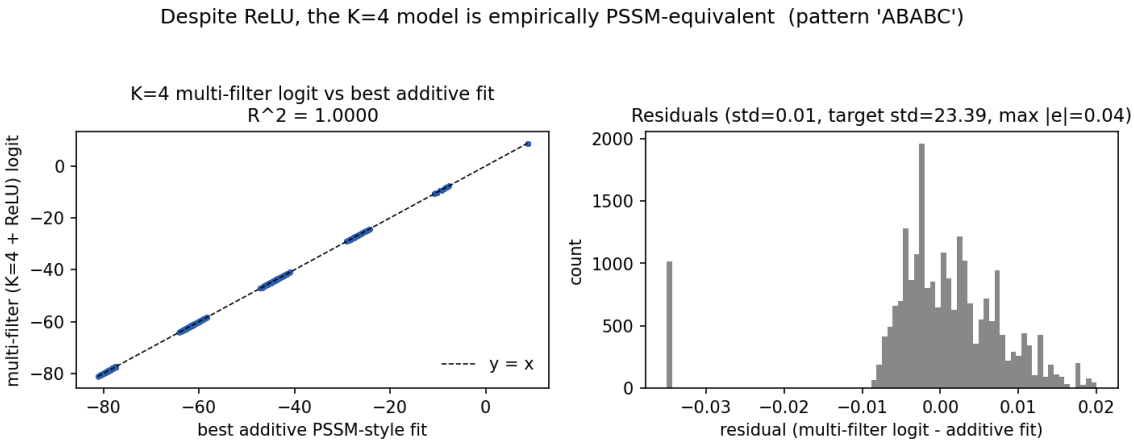


Figure 6: Multi-filter ($K = 4$ + ReLU) logit vs the best additive PSSM-style fit. Left: scatter on the identity line. Right: residual histogram, all within $\pm 0.04$ on logits whose spread is $\approx 47$.

## 6 The penalty table as an algorithmic data structure

Reading pen as a *preprocessing of the pattern* puts it on the same shelf as *sp* and *bc*. Table 1 summarises the comparison.

The penalty table is **strictly richer than** *bc*: it is position-aware (the same wrong character at position 0 vs position 4 of `ABABC` receives penalties 4.77 vs 7.49). It is also **strictly richer than the ground-truth PWM**, whose penalties are binary $\{0, 1\}$; pen is graded by the training distribution. It is **strictly poorer than** *sp* in one important respect: it is *stateless*. There is no way to encode "after a partial match of length $j$, shift by $j - sp[j-1]$" in a per-position-per-character lookup. *sp* orchestrates the *control flow* of a scan; pen scores complete windows.

This observation will let us, in Section 8, recover *Boyer–Moore-style* shift behaviour from pen but not *KMP-style*.

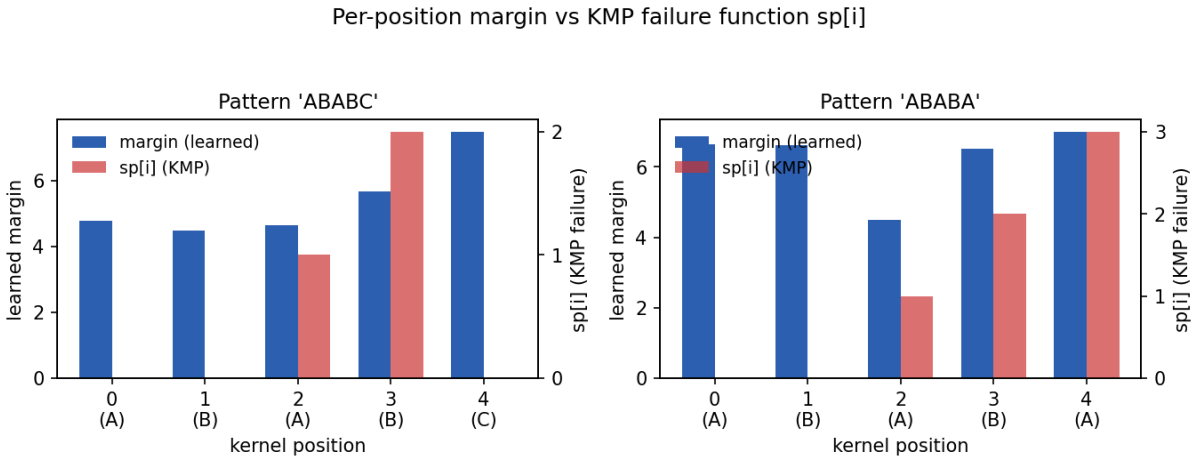


Figure 7: Per-position learned margin (blue, left axis) vs $sp[i]$ (red, right axis). On `ABABA` position 2 has both the first non-zero $sp$ and the lowest margin. On `ABABC` the relationship does not hold.

| | KMP $sp$ | BM bad-character $bc$ | CNN penalty pen |
|---|---|---|---|
| Indexed by | pattern position $i$ | character $c$ | (position $i$, character $c$) |
| Range | $\{0, \ldots, i\}$ | $\{-1, 0, \ldots, m-1\}$ | $\mathbb{R}_{\geq 0}$ |
| Construction time | $\mathcal{O}(m)$ | $\mathcal{O}(m + \lvert\Sigma\rvert)$ | $\mathcal{O}(m \cdot \lvert\Sigma\rvert)$ from $w$ |
| Construction source | $P$ alone | $P$ alone | $P$ + training distribution |
| Storage | $\mathcal{O}(m)$ | $\mathcal{O}(\lvert\Sigma\rvert)$ | $\mathcal{O}(m \cdot \lvert\Sigma\rvert)$ |
| Query | $\mathcal{O}(1)$ shift on mismatch | $\mathcal{O}(1)$ shift on mismatch | $\mathcal{O}(1)$ score lookup per char |
| Drives | control flow of the scan | control flow of the scan | additive scoring of windows |

Table 1: The penalty table compared with KMP's failure function and Boyer–Moore's bad-character rule, as preprocessing data structures.

## 7 Approximate / k-mismatch matching

Identity (4) makes the connection to approximate matching mechanical. A $k$-mismatch matcher accepts windows with at most $k$ character disagreements; in our setting the disagreement at position $i$ contributes $\text{pen}[i, W[i]]$ to a cumulative cost. Choosing a threshold $\tau$ on the cumulative penalty:

- $\tau = 0$ is exact matching (only the recovered pattern survives);
- $\tau \in [0, \min_{i, c \neq P_i} \text{pen}[i, c])$ is still exact, but resilient to floating-point noise;
- $\tau \geq \min_i \text{margin}[i]$ admits some 1-mismatch windows;
- $\tau \to \infty$ admits everything.

The threshold knob on a CNN classifier is therefore, on this task, a threshold on a generalised Hamming distance to the recovered pattern, with weights given by pen. This connects the experiment directly to the classical literature on FFT-based exact and wildcard matching [4], where the same convolution-as-string-matching identity is the core algorithmic ingredient.

**Algorithm 1** Naive matcher (matcher A).

```
1: function NaiveScan(T, pen, τ)
2:     matches ← ∅
3:     for s ← 0 to |T| − m do
4:         S ← 0
5:         for i ← 0 to m − 1 do                    ▷ always m lookups
6:             S ← S + pen[i, T[s + i]]
7:         end for
8:         if S ≤ τ then matches ← matches ∪ {s}
9:         end if
10:    end for
11:    return matches
12: end function
```

## 8 Direction 2: a penalty-driven dynamic-stride scan

We use pen as actual algorithmic preprocessing. From it we build, in $\mathcal{O}(m \cdot |\Sigma|)$ time, a generalised Boyer–Moore bad-character shift table:

$$\text{shift}[i, c] \;=\; \max\Big(1,\; i \;-\; \max\{\, k \le i \mid \text{pen}[k, c] \le \eta \,\}\Big), \tag{5}$$

with $\text{shift}[i, c] = i + 1$ if no admissible $k$ exists. With $\eta = 0$ this is exactly the classical Boyer–Moore bad-character rule applied to the recovered pattern; with $\eta > 0$ admissibility is graded by the learned penalty values, yielding a "soft" approximate variant.

For our trained filter on `ABABC` the resulting shift table is

| | $A$ | $B$ | $C$ | $D$ |
|---|---|---|---|---|
| pos 0 | 1 | 1 | 1 | 1 |
| pos 1 | 1 | 1 | 2 | 2 |
| pos 2 | 1 | 1 | 3 | 3 |
| pos 3 | 1 | 1 | 4 | 4 |
| pos 4 | 2 | 1 | 1 | 5 |

which agrees character-for-character with classical BM applied to `ABABC`, and which we computed end-to-end from the CNN's weights.

We compare three matchers on the same long text. All of them score windows by the cumulative penalty $\sum_i \text{pen}[i, T[s + i]]$ and accept iff this sum is at most a threshold $\tau$, but they differ in how they orchestrate the scan:

**A naive** Always do all $m$ penalty lookups per text position (Algorithm 1).

**B early-exit** Same as A, but stop the per-position accumulation as soon as the running sum exceeds $\tau$. Stride is still 1 but the per-position cost is data-dependent (Algorithm 2).

**C early-exit + BM shift** Same as B, plus a Boyer–Moore-style shift (5) when an early exit fires (Algorithm 3).

**Empirical results.** We ran the three matchers on a single 20 000-character text containing 80 planted matches and 200 deliberate near-misses, producing 112 true match positions after KMP labelling (the extra 32 come from accidental boundary matches). Threshold $\tau = 4.96$, which equals the pattern logit at $b = 0$ and corresponds to threshold 0 on the trained model's logit. Cost is reported in penalty-table lookups (the elementary operation common to all three matchers); wall time is on a single CPU core.

**Algorithm 2** Early-exit matcher (matcher B).

```
function EarlyExitScan(T, pen, τ)
    matches ← ∅
    for s ← 0 to |T| − m do
        S ← 0;   rejected ← false
        for i ← 0 to m − 1 do
            S ← S + pen[i, T[s + i]]
            if S > τ then rejected ← true; break
            end if
        end for
        if ¬ rejected then matches ← matches ∪ {s}
        end if
    end for
    return matches
end function
```

| matcher | ops | ops/pos | wall (s) | TP | FP | FN | prec. | recall | F1 | speedup |
|---|---|---|---|---|---|---|---|---|---|---|
| A naive | 99 980 | 5.00 | 0.0171 | 112 | 71 | 0 | 0.612 | 1.000 | 0.759 | 1.00× |
| B early-exit | 36 351 | 1.82 | 0.0079 | 112 | 71 | 0 | 0.612 | 1.000 | 0.759 | **2.75**× |
| C early-exit + BM | 24 634 | 1.23 | 0.0076 | 112 | 71 | 0 | 0.612 | 1.000 | 0.759 | **4.06**× |

Table 2: Cost and detection metrics on a 20 000-character text with 112 true matches. The early-exit + BM matcher visits the equivalent of 1.23 lookups per text position, vs 5.00 for the naive scan — a 4.06× reduction with no loss of detection quality.

Detection quality is identical across the three matchers (recall 1.0; the same 71 one-mismatch survivors at this $\tau$, the same precision and F1). Reducing $\tau$ would eliminate them at the cost of recall on a few boundary cases. The speedup is therefore "free" in the sense that it does not trade off detection quality.

**What this prototype shows, in algorithmic terms.**

1. The penalty table pen, computed in $\mathcal{O}(m \cdot |\Sigma|)$ from a trained filter, *is* a usable preprocessing structure: it powers both the per-position cost reduction (early exit) and the inter-position shift (BM-style).
2. With $\eta = 0$ the prototype recovers classical Boyer–Moore on the pattern recovered from the filter — "training a CNN and reading the bad-character rule off its weights" is a complete pipeline.
3. With $\eta > 0$ one obtains a *soft* or approximate variant whose admissibility map is *learned from data* — a direction the literature review collected in this project identifies as unexplored under exactly this framing.

## 9 Discussion: why the failure function does not emerge

The penalty table is *stateless*: it scores each window independently of the others. The failure function *sp* is *stateful*: it expresses how the result of a partial match informs the next position to look at. The two structures live in different computational regimes.

This matches the formal-language-theoretic placement of CNNs. Merrill [5] shows that shallow 1D CNNs recognise at most *strictly local* languages, a proper subset of the regular languages. KMP's correctness relies on a deterministic finite automaton over the pattern; that DFA is regular

**Algorithm 3** Early-exit + Boyer–Moore-style shift (matcher C).

```
function EarlyExitBMScan(T, pen, shift, τ)
    matches ← ∅;   s ← 0
    while s + m ≤ |T| do
        S ← 0;   j ← −1
        for i ← 0 to m − 1 do
            c ← T[s + i]
            S ← S + pen[i, c]
            if S > τ then j ← i; break
            end if
        end for
        if j = −1 then
            matches ← matches ∪ {s}
            s ← s + 1
        else
            s ← s + shift[j, T[s + j]]
        end if
    end while
    return matches
end function
```

but not strictly local. A pure CNN therefore cannot, in principle, implement KMP's shift logic in the weights, and our experiments show that, in practice, it does not even try. It instead solves the easier scoring problem and lets the threshold do the rest.

The penalty table is what naturally fits in a stateless template-matching machine. Recovering an *sp*-like role in the weights would require adding state: a recurrent layer, attention, or an explicit memory mechanism. Neural Algorithmic Reasoning work [6] has shown that GNN-based learners can be trained to imitate KMP, but at significant cost in performance and sample complexity, and crucially via message-passing rather than convolution.

Where genomics CNNs explicitly use filters as Position Weight Matrices [7, 8], our identity (4) formalises and slightly extends this view: not only do the filters *look like* PWMs, they *provably are* PSSMs in their entire input–output behaviour for any one-hot input.

## 10 Limitations and outlook

**Single pattern, small alphabet, short kernel.** The clean separability we observe is partly an artefact of $|\Sigma| = 4$ and $m = 5$. Whether the additive identity continues to hold for natural-language alphabets and longer kernels is a question of how often ReLU clips, not of principle, but worth checking.

**Counter-example to the multi-filter collapse.** Designing a task on which a $K > 1$ ReLU CNN provably uses its non-linearity to compute something not expressible as a per-(position, character) lookup would isolate the regime in which the additive identity fails. The natural candidate is a task whose label depends on long-range correlations the kernel cannot see at once, e.g. multiple exact patterns combined by an OR (Aho–Corasick territory).

**Soft admissibility ($\eta > 0$).** Our prototype uses $\eta = 0$ and so reduces to classical Boyer–Moore. Tuning $\eta > 0$ to admit one-mismatch windows by penalty cost gives a learned-from-data approximate-matching algorithm whose theoretical worst-case behaviour we have not analysed.

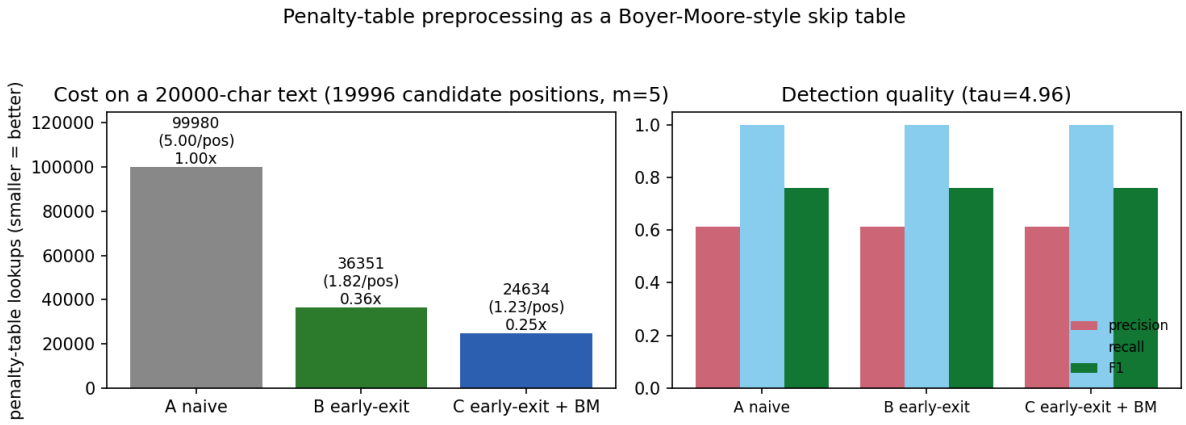


Figure 8: Cost (in penalty-table lookups, left) and detection quality (right) for the three matchers of Algorithms 1–3 on a 20 000-character text. Detection quality is identical across A–C; the elementary-operation count drops from 5.00/pos to 1.23/pos.

**State-augmented models.** A small recurrent gating layer over the penalty-table output is the smallest architectural change that could plausibly recover an *sp*-like role in the weights. We have not tested it.

## 11 Conclusion

The relationship between 1D CNNs and exact pattern matching looks different depending on which preprocessing data structure one uses as the reference. Against KMP's failure function *sp*, the CNN comes up short: the architecture is stateless and cannot recover a sequential shift table. Against the Boyer–Moore bad-character rule, the CNN does *better* than the analogue: its trained filter implicitly encodes a *position-aware, real-valued* bad-character table that generalises *bc* both in its index space (a matrix instead of a vector) and in its range (real numbers instead of integers). We made this explicit by deriving the penalty-table identity (4), verifying it numerically to floating-point precision, and using it as the preprocessing of an actual matching algorithm that achieves a 4.06× reduction in elementary operations on a long text. Exact matching, in this sense, is just the limit case of the more general approximate-matching scoring that the same penalty table naturally supports.

**Reproducibility.** The full code is JAX-only and runs in under two minutes on a single CPU core. Each experiment is driven by a stand-alone script:

```
uv run python experiment1.py     # single filter, ABABC (Direction 1)
uv run python experiment2.py     # K=4 filters, ABABC
uv run python experiment3.py     # single filter, ABABA (auto-overlap)
uv run python experiment4.py     # the penalty-table identity
uv run python experiment5.py     # Direction 2 prototype (this section)
uv run python make_figures.py    # regenerates every figure
```

The companion `LITERATURE_REVIEW.md` collects the state-of-the-art on both directions of the analogy, with ∼50 references, and identifies in particular the gap that motivates the Direction 2 prototype above.

## References


[1] D. E. Knuth, J. H. Morris, V. R. Pratt. *Fast pattern matching in strings.* SIAM Journal on Computing, 6(2):323–350, 1977.

[2] R. S. Boyer, J. S. Moore. *A fast string searching algorithm.* Communications of the ACM, 20(10):762–772, 1977.

[3] D. Gusfield. *Algorithms on strings, trees, and sequences: computer science and computational biology.* Cambridge University Press, 1997.

[4] P. Clifford, R. Clifford. *Simple deterministic wildcard matching.* Information Processing Letters, 101(2):53–54, 2007.

[5] W. Merrill. *Sequential neural networks as automata.* Proceedings of the ACL Workshop on Deep Learning and Formal Languages, 2019.

[6] P. Veličković, A. P. Badia, D. Budden, R. Pascanu, A. Banino, M. Dashevskiy, R. Hadsell, C. Blundell. *The CLRS algorithmic reasoning benchmark.* Proceedings of ICML, 2022.

[7] B. Alipanahi, A. Delong, M. T. Weirauch, B. J. Frey. *Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning.* Nature Biotechnology, 33(8):831–838, 2015.

[8] P. K. Koo, S. R. Eddy. *Representation learning of genomic sequence motifs with convolutional neural networks.* PLOS Computational Biology, 15(12):e1007560, 2019.

[9] C.-C. J. Kuo et al. *Convolutional neural networks demystified: a matched filtering perspective-based tutorial.* IEEE Signal Processing Magazine, 40(1), 2023.

[10] S. Cao et al. *SeerNet: predicting CNN feature-map sparsity through low-bit quantization.* Proceedings of CVPR, 2019.

[11] W. Hua, Y. Zhou, C. De Sa, Z. Zhang, G. E. Suh. *Channel gating neural networks.* Proceedings of NeurIPS, 2019.

[12] T. Verelst, T. Tuytelaars. *Dynamic convolutions: exploiting spatial sparsity for faster inference.* Proceedings of CVPR, 2020.